

Week 38, Tuesday Session

Flipped classroom: your questions, then AdaGrad, RMSProp, Adam and stochastic gradient descent in practice

Bring your laptop – we work in `week38tuesday.ipynb`

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

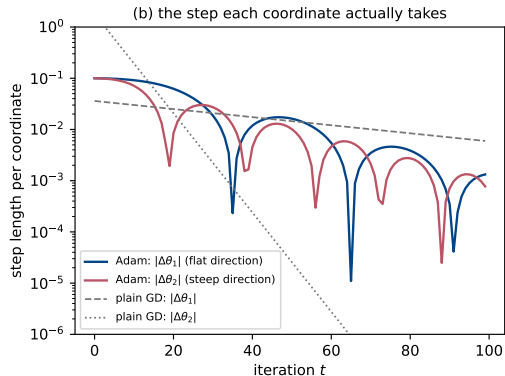
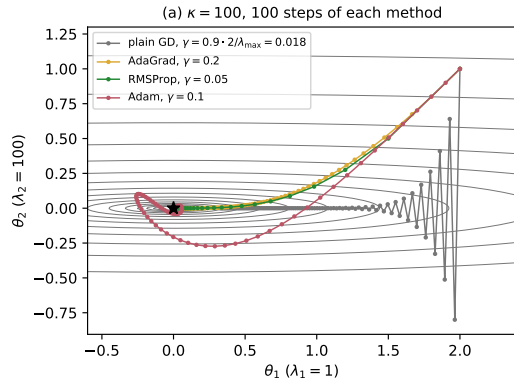
How today works

The format (as every Tuesday)

Mondays are lectures; Tuesdays belong to your questions and to concrete cases. You have heard the lecture and skimmed Sections 4.7–4.14.

1. **First ~20 min:** your open questions from Monday's lecture and from the reading, discussed in plenum. Automatic differentiation questions are welcome here – it was new on Monday and part e) of Project 1 uses it.
2. **Rest of the double session:** *two* cases only – the adaptive optimisers in your own code on the Runge function, and stochastic gradient descent with minibatches, epochs and schedules – worked in pairs or small groups from `week38tuesday.ipynb`. Deep beats broad: the goal is that you leave with one step function holding five optimisers, an sgd loop, and the habit of scanning the learning rate *per method*. Case 1 is part f) of Project 1 and exercises 2–3; Case 2 is part h) and exercise 4.
3. **Last 5 min:** wrap-up, exercise deadline, and what next Monday builds on.

Picking up from Monday: γ means something different to each method



On the $\kappa = 100$ bowl, plain gradient descent at $0.9 \gamma_{\max}$ zigzags down the steep direction and starves the flat one; the adaptive methods normalise each coordinate by its own gradient history and set off diagonally at about γ per step – for them γ is a *distance*, not a multiple of the gradient. Today you build the step function that produced this figure, measure each method's window of usable γ on the Runge problem, and then trade the full gradient for a minibatch.

Case 1: AdaGrad, RMSProp and Adam in your own code, on the Runge function

Goal: part f) of Project 1 in miniature (notebook, Case 1; seed 2026 gives exactly the lecture numbers). Data $f(x) = 1/(1 + 25x^2)$ on $[-1, 1]$, $n = 100$, $\sigma = 0.1$, standardised columns, centred y .

1. **The problem:** Hessian eigenvalues, κ , $2/\lambda_{\max}$ at degrees 6 and 10.
2. **One step function, five optimisers:** fill in `optimiser_step` from Eqs. (4.10), (4.28), (4.42)–(4.45), (4.47)–(4.48), (4.51)–(4.55). The gradient is an argument; the state dictionary carries v , r , m .
3. **Degree 6:** all five from $\theta_0 = 0$; iterations to an excess cost $C(\theta_k) - C(\hat{\theta})$ of 10^{-4} and 10^{-8} . Which method never gets there?
4. **The scan:** γ from 10^{-3} to 1, sixteen values, for each method: the window of usable γ and the best value. Compare with $2/\lambda_{\max}$ and with $2(1 + \beta)/\lambda_{\max}$.
5. **Degree 10:** momentum and Adam for 30 000 iterations; then Ridge with $\lambda = 10^{-3}$; then one Newton step with $\frac{2}{n}X^T X$. Is Ridge cheating?

Checkpoints (plenary after ~ 30 min)

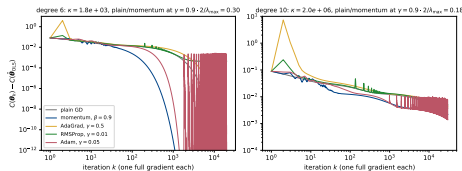
Degree 6, $\kappa = 1849$, iterations to 10^{-8} : plain 7346, momentum 670, AdaGrad 12 156, Adam 1062, RMSProp never; Adam converges for every γ in the scan. Degree 10: all stall near 10^{-3} ; Ridge 10^{-3} ($\kappa = 4975$): Adam 1210; Newton: one step, 2×10^{-10} .

Case 1: the code you should end up with

```
def optimiser_step(method, theta, g, state, t, gamma, beta=0.9, rho=0.99,
                  beta1=0.9, beta2=0.999, eps=1e-8):
    """One update of theta from the gradient g at step t = 1, 2, ...; state carries the running
        quantities."""
    if method == "plain":
        return theta - gamma * g, state
    if method == "momentum":
        state["v"] = v = beta * state.get("v", 0.0) + gamma * g # Eq. (4.28)
        return theta - v, state
    if method == "adagrad":
        state["r"] = r = state.get("r", 0.0) + g * g # Eq. (4.42)
        return theta - gamma * g / (np.sqrt(r) + eps), state # Eq. (4.45)
    if method == "rmsprop":
        state["r"] = r = rho * state.get("r", 0.0) + (1.0 - rho) * g * g # Eq. (4.47)
        return theta - gamma * g / (np.sqrt(r) + eps), state # Eq. (4.48)
    if method == "adam":
        state["m"] = m = beta1 * state.get("m", 0.0) + (1.0 - beta1) * g # Eq. (4.51)
        state["r"] = r = beta2 * state.get("r", 0.0) + (1.0 - beta2) * g * g # Eq. (4.52)
        m_hat, r_hat = m / (1.0 - beta1**t), r / (1.0 - beta2**t) # Eq. (4.54)
        return theta - gamma * m_hat / (np.sqrt(r_hat) + eps), state # Eq. (4.55)
```

The loop around it (optimise in the notebook) is last week's `gradient_descent` with the update replaced by one call to this function. Keep the structure for Project 1: one cost, one gradient, one step function, and the epoch loop of Case 2 around it.

Case 1: what you should find



The scan at degree 6 (iterations to 10^{-6})

method	usable γ	best
plain	0.10–0.25	5960 at 0.25
momentum	0.01–0.63	200 at 0.63
AdaGrad	0.04–1	7390 at 0.16
RMSProp	none	–
Adam	0.001–1	590 at 0.10

- ▶ Plain descent: a window of a factor 2.5 below $2/\lambda_{\max} = 0.334$. Momentum's window ends at $2(1 + \beta)/\lambda_{\max} = 0.63$ – exactly.
- ▶ AdaGrad works for any γ above 0.04 and is *slower than plain descent* at every one of them: its steps only ever shrink.
- ▶ RMSProp at a constant γ never reaches 10^{-6} : near the minimum $r \approx g^2$ and the step is $\approx \gamma \text{sign}(g)$.
- ▶ Adam converges over three decades of γ , best at 0.1 – and still *twice* as fast at its best γ as at $\gamma = 1$.
- ▶ Degree 10: momentum 2.2×10^{-3} , Adam 1.4×10^{-3} after 30 000 iterations. Ridge 10^{-3} : momentum 1330, Adam 1210 iterations to 10^{-8} – of a *different* problem, with coefficients $(-0.06, -1.08, 0.14, 1.33, \dots)$ against OLS's $(-0.11, -3.09, 0.43, 13.4, \dots, -26.5, \dots)$.

Case 1: do you have the insight?

1. Adam converged for every learning rate in your scan, from 10^{-3} to 1, while plain gradient descent had a window of a factor of three. What makes Adam insensitive to γ – and does that mean γ does not matter for Adam?
2. RMSProp at a constant γ never reached 10^{-6} , on a convex quadratic. What is its step near the minimum, and what would you change?
3. AdaGrad needed more iterations than plain gradient descent at degree 6. Which property of AdaGrad is responsible, and on what kind of problem does that same property become an advantage?
4. At degree 10 every first-order method stalled at an excess cost of about 10^{-3} , but Ridge with $\lambda = 10^{-3}$ converged in about a thousand iterations and a single Newton step hit the OLS solution. Is Ridge cheating? What did Newton use that the others did not have?

Case 1: do you have the insight?

1. Adam converged for every learning rate in your scan, from 10^{-3} to 1, while plain gradient descent had a window of a factor of three. What makes Adam insensitive to γ – and does that mean γ does not matter for Adam?
2. RMSProp at a constant γ never reached 10^{-6} , on a convex quadratic. What is its step near the minimum, and what would you change?
3. AdaGrad needed more iterations than plain gradient descent at degree 6. Which property of AdaGrad is responsible, and on what kind of problem does that same property become an advantage?
4. At degree 10 every first-order method stalled at an excess cost of about 10^{-3} , but Ridge with $\lambda = 10^{-3}$ converged in about a thousand iterations and a single Newton step hit the OLS solution. Is Ridge cheating? What did Newton use that the others did not have?

What we hope you say

(1) Adam divides by $\sqrt{\hat{v}}$: the step is at most γ per coordinate whatever the gradient, so no γ can violate Eq. (4.20). It still matters – best was six times faster than worst – it just cannot make the run *diverge*. (2) $\gamma \text{sign}(\mathbf{g})$: a fixed-length step, so the iterate rattles at radius $\sim \gamma$. Decay γ (Eq. 4.40), or add the first moment – which is Adam. (3) r_t never forgets: $\gamma/\sqrt{r_t}$ decays like $1/\sqrt{t}$ whether or not the minimum is near – too slow on a smooth convex problem, right for *sparse* features whose $r_{t,j}$ stays small until they appear. (4) Ridge solves a different problem, Eq. (4.16), with κ 400 times smaller and different coefficients – bias for variance, Section 3.8, possibly the better fit (part i). Newton used the *full* Hessian, off-diagonal entries included; the adaptive methods see only the diagonal, and the valley of a correlated polynomial basis is not axis-aligned.

Case 2: stochastic gradient descent

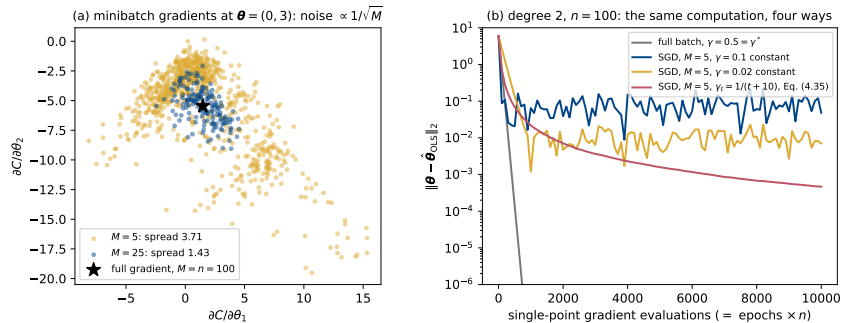
Goal: part h) of Project 1 in miniature (notebook, Case 2). Data $f(x) = 2 - x + 5x^2$ on $[-2, 2]$, $n = 100$, $\sigma = 0.5$, degree 2 ($\kappa = 1.3$) and degree 5 ($\kappa = 520$), standardised columns, centred y .

1. **The minibatch gradient is a noisy, unbiased estimate:** at $\theta = (0, 3)$, draw many minibatches of size $M \in \{1, 5, 25\}$ and compare the mean and spread of their gradients with the full one.
2. **The loop:** write `sgd` – reshuffle every epoch, split into minibatches, one `optimiser_step` per minibatch, record θ after every epoch. Plain SGD, $M = 5$, $\gamma = 0.1$ and 0.02 , 100 epochs: distance to $\hat{\theta}$ after 1, 10, 100 epochs. Does it converge?
3. **Batch size:** $M \in \{1, 5, 20, 100\}$ at $\gamma = 0.1$, on both data sets, counting *gradient evaluations*. What happens with $M = 1$ at degree 5, and which Hessian explains it?
4. **Schedules:** $\gamma_t = t_0/(t + t_1)$, Eq. (4.40), with $(1, 10)$ on both data sets; repair it at degree 5. Reproduce `week38_sgd.pdf`.
5. **Adam with minibatches:** degree 5, $M = 5$: plain 0.1, momentum 0.05, Adam 0.01 and 0.05, RMSProp 0.01. Which wins, which diverges, and could you have predicted the divergence?

Checkpoints (seed 2026)

Spread 8.5 / 3.7 / 1.5 for $M = 1, 5, 25$. Degree 2, $M = 5$, $\gamma = 0.1$: 5×10^{-2} after 100 epochs and not improving; $(1, 10)$: 5×10^{-4} . Degree 5: $M = 1$ diverges ($2|x_i|^2$ up to 86, so $\gamma < 0.023$); $(1, 10)$ freezes at 3.0, $(20, 200)$ reaches 0.25; Adam 0.05: 0.14; momentum 0.05: 10^{19} .

Case 2: what you should find



(a) The cloud is centred on the full gradient and its spread halves when M is quadrupled: unbiased, noise $\propto 1/\sqrt{M}$ (Section 2.5). (b) Per gradient evaluation SGD is ahead for a few hundred evaluations, then *rattles* at a radius set by γ and the noise; full-batch descent at γ^* overtakes at about 1000 evaluations on this $\kappa = 1.3$ problem – on the $\kappa = 520$ problem it is still at 2.0 after 10 000 while $M = 5$ is at 0.13. The schedule keeps descending where the constant rate stalls, *if* it decays slowly enough: $(1, 10)$ is at $\gamma = 5 \times 10^{-4}$ after 100 epochs and froze at degree 5; $(20, 200)$ has the same γ_0 and gets to 0.25.

Case 2: do you have the insight?

1. Plain SGD with $M = 5$ at a constant $\gamma = 0.1$ sat at a distance of 5×10^{-2} from the minimum for ninety epochs. Has it converged? What is it doing, and what sets the size of the cloud?
2. Full-batch gradient descent beat every SGD variant per gradient evaluation on the degree-2 data. Is that the general rule? What is different about a data set with $n = 10^6$?
3. The schedule $(t_0, t_1) = (1, 10)$ worked on the degree-2 data and froze on the degree-5 data. Which of the two Robbins–Monro conditions did it violate in practice, and how did you repair it?
4. With $M = 1$ at $\gamma = 0.1$ the degree-5 run diverged although $\gamma < 2/\lambda_{\max}$ for the full Hessian. Which Hessian matters for a minibatch step?

Case 2: do you have the insight?

1. Plain SGD with $M = 5$ at a constant $\gamma = 0.1$ sat at a distance of 5×10^{-2} from the minimum for ninety epochs. Has it converged? What is it doing, and what sets the size of the cloud?
2. Full-batch gradient descent beat every SGD variant per gradient evaluation on the degree-2 data. Is that the general rule? What is different about a data set with $n = 10^6$?
3. The schedule $(t_0, t_1) = (1, 10)$ worked on the degree-2 data and froze on the degree-5 data. Which of the two Robbins–Monro conditions did it violate in practice, and how did you repair it?
4. With $M = 1$ at $\gamma = 0.1$ the degree-5 run diverged although $\gamma < 2/\lambda_{\max}$ for the full Hessian. Which Hessian matters for a minibatch step?

What we hope you say

(1) No. It has *equilibrated*: a random walk pulled back by the gradient, in a cloud of radius $\propto \sqrt{\gamma} \sigma_g / \sqrt{M}$. Smaller γ , larger M , or a decaying γ_t shrink it; a longer run does not. (2) No. On a small benign problem the exact gradient is cheap and converges geometrically, Eq. (4.26). With $n = 10^6$ one full gradient costs 10^4 minibatch steps of $M = 100$: SGD is 10^4 updates ahead before gradient descent has moved, and never needed the data set in memory. (3) $\sum_t \gamma_t = \infty$ – in principle satisfied, in practice not: γ_t was 5×10^{-4} by epoch 100 and the sum over the run was too small to cross a $\kappa = 520$ valley. Keep $\gamma_0 = t_0/t_1$ and make both larger, so that the decay starts later. (4) The Hessian of the *minibatch* cost: for one point it is $2x_i x_i^T$ with eigenvalue $2\|x_i\|^2$, up to 86 at degree 5, so $\gamma < 0.023$. The full-batch bound is an average; a single step sees one batch. Adaptive methods normalise this away – one reason Adam is the default with minibatches – and momentum makes it worse: $\gamma/(1 - \beta)$.

Wrap-up, deadlines, and Monday

- ▶ **What you should own now:** one step function with five optimisers and one gradient (by hand or `jax.grad`); an sgd loop with minibatches, epochs and a schedule; the habit of scanning γ *per method*; and one sentence each on what AdaGrad, RMSProp and Adam do to the step, what the minibatch noise is, and why a constant learning rate cannot converge.
- ▶ **Weekly exercises** (`exercisesweek38.ipynb`): derivations for the adaptive methods (bias correction, the first Adam step, scale invariance), Cases 1 and 2 as a write-up on the exercise data, and the Runge problem at degree 10. Deadline **Friday September 18 at midnight**.
- ▶ **Project 1**, parts f) and h), ask for exactly today's machinery on the Runge function; part g) (Lasso) needs one more ingredient, the subgradient of $|\theta|$ – next Monday.
- ▶ **Next Monday (week 39):** Lasso by gradient methods (Section 3.9, part g) and logistic regression (Chapter 5) – the first non-quadratic cost, where `jax.grad` earns its keep.

Before you leave

One line in an email to us: the muddiest point of the week – what should Monday's first ten minutes revisit?